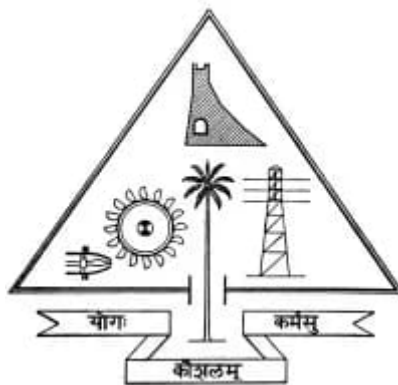


DIFFICULTY-CURVE CONDITIONED PROCEDURAL LEVEL GENERATION FOR GAMES USING DEEP GENERATIVE SEQUENCE MODELS

*Mini Project Report submitted in partial fulfillment of the requirements for the
award of the degree of Master of Computer Applications of the APJ Abdul
Kalam Technological University*

Submitted by

**AJITH K S
(TCR25MCA-2008)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

JULY 2026

ABSTRACT

Procedural Content Generation (PCG) for games sits at the intersection of generative modeling and game design automation. Most procedural level generators produce content conditioned on a single static difficulty label or random seed, treating each level as an independent artifact. This ignores pacing — the designed rise and fall of challenge across a sequence of levels that keeps players engaged. Recent benchmarking work confirms this gap: systems that adapt in real time must detect shifts in player skill without causing disengagement, and standard PCG evaluation frameworks continue to rely on testbeds built around single-level quality, diversity, and controllability metrics rather than pacing-aware sequence generation. Existing difficulty-aware generators tend to focus on estimating or balancing difficulty within a single level rather than orchestrating a coherent trajectory across a run of levels. This project proposes a trajectory-conditioned generative pipeline that takes a designer-drawn difficulty curve (e.g., easy → spike → recovery → ramp) as input and outputs a sequence of tile-based levels matching that curve point-by-point. The system combines: (1) a difficulty estimator trained on measurable proxies such as path length and enemy density, (2) a generative model (transformer or GAN over level tiles) conditioned on a per-level target difficulty rather than a single global tag, and (3) an automated playability validator ensuring every generated level is solvable before being surfaced. A designer-facing curve editor treats the trajectory itself as the primary creative object, rather than difficulty being a byproduct of content generation. Prior mathematical and curve-fitting approaches to PCG difficulty balancing describe solutions aimed at automatic level creation with balanced difficulty, though few recent works address difficulty balancing directly and many rely on computationally intensive evolutionary or graph-based techniques. Hybrid approaches combining Wave Function Collapse with genetic search have improved structural coherence by extending constraint-based generation with genetic search operators, but still generate levels independently rather than as a pacing-aware sequence. Recent surveys on LLM-integrated PCG point toward deep integration of large language models, reinforcement learning, and symbolic constraint solvers for co-creative, dynamically adaptable content across genres; this project operationalizes that direction at a narrower, postgraduate-feasible scope — difficulty-trajectory conditioning rather than open-ended multi-modal generation. Unlike single-level difficulty estimators, the core contribution here is explicit sequence-level conditioning validated against a human-drawn pacing curve, closing the gap between per-level difficulty estimation and designed player-experience pacing.

CONTENTS

List of Abbreviations	iv
1 INTRODUCTION	1
2 LITERATURE REVIEW	4
2.1 Procedural Dungeon Generation for a 2D Top-Down Game . .	4
2.2 Experience-Driven PCG via Reinforcement Learning: A Super Mario Bros Study	5
2.3 Fast Game Content Adaptation Through Bayesian-Based Player Modelling	5
2.4 Finding Game Levels with the Right Difficulty in a Few Trials Through Intelligent Trial-and-Error	6
2.5 Large Language Models and Games: A Survey and Roadmap	6
2.6 Summary and Research Gap	7
3 ENVIRONMENTAL STUDY	10
3.1 System Configuration	10
3.1.1 Hardware Requirements	11
3.1.2 Software Requirements	11
3.1.3 Software Requirements for Development	13
3.2 Software Specification	13
3.2.1 Python 3.10	14
3.2.2 Visual Studio Code	14
3.2.3 PyTorch	14
3.2.4 Pygame	15
3.2.5 A* Pathfinding / Playability Validator	15
3.2.6 Flask	15
3.2.7 Git	16
4 SYSTEM ANALYSIS	17
4.1 Requirements Analysis	17
4.2 Existing System	18
4.3 Limitations of Existing System	18
4.4 Proposed System	19
4.5 Advantages of Proposed System	20
4.6 Functional Requirements	21
4.6.1 Level Representation Setup	21
4.6.2 Difficulty Estimator	21
4.6.3 Trajectory-Conditioned Generator	21
4.6.4 Playability Validator	22
4.6.5 Designer Trajectory Editor	22
4.6.6 Interactive Demo Interface	22
4.7 Non-Functional Requirements	23

4.7.1	Scalability	23
4.7.2	Usability	23
4.7.3	Reliability	23
4.7.4	Maintainability	23
4.7.5	Performance	24
4.8	Feasibility Study	24
4.8.1	Technical Feasibility	24
4.8.2	Operational Feasibility	24
4.8.3	Economic Feasibility	25
5	SYSTEM DESIGN	26
5.1	System Architecture	26
5.2	Data Flow Diagram (DFD)	28
5.2.1	DFD Level 0	28
5.2.2	DFD Level 1	28
5.3	List of Modules	29
5.3.1	Web Engine and User Interface	30
5.3.2	Trajectory-Conditioned Sequence Generator	30
5.3.3	A* Pathfinding Playability Validator	30
5.3.4	Selective Cumulative Session DDA Engine	31
5.3.5	Game Session and Arcade Manager	31
5.3.6	Interactive Play and Telemetry Module	31
	Bibliography	33

LIST OF ABBREVIATIONS

AI	– Artificial Intelligence
DDA	– Dynamic Difficulty Adjustment
GAN	– Generative Adversarial Network
IT&E	– Intelligent Trial-and-Error
LLM	– Large Language Model
MDP	– Markov Decision Process
PCG	– Procedural Content Generation
RAM	– Random Access Memory
RL	– Reinforcement Learning
SRS	– Software Requirements Specification
UI	– User Interface

CHAPTER 1

INTRODUCTION

Procedural Content Generation (PCG) refers to the algorithmic creation of game content such as levels, terrain, items, and rules, with limited or no direct human authorship. PCG has become a central technique in modern game development because it reduces authoring cost, increases replayability, and allows a single game to present a practically unlimited number of level variations to different players. Within PCG, level generation for platformers, dungeons, and maze-like environments has received particular attention because such levels can be represented compactly as sequences or grids of tiles, making them tractable for search-based, grammar-based, and machine-learning-based generators.

A closely related and equally important problem is difficulty management. Players differ widely in skill, and a single static difficulty setting – easy, medium, or hard – rarely suits every player throughout an entire play session. This observation motivated the field of Dynamic Difficulty Adjustment (DDA), which studies how a game can observe player performance and adapt its challenge in response, with the goal of keeping the player in a state of engagement rather than boredom or frustration. Early DDA systems such as Hamlet adjusted numeric attributes like ammunition and health using probabilistic inventory models, while later systems used reinforcement learning and fuzzy logic to tune the behaviour of non-player characters to the estimated skill of the player. More recent work has gone further and shown that difficulty can be adjusted structurally, by generating or selecting entire level segments rather than only tweaking numeric parameters.

Despite this progress, most procedural level generators and most difficulty-

aware generators still treat each level as an independent artefact conditioned on a single, static difficulty label or a random seed. This is true both of classic search-based generators and of more recent machine-learning generators trained with reinforcement learning or generative adversarial networks. Consequently, while an individual level can be tuned to be easy, medium, or hard, the generator has no explicit notion of *pacing* – the deliberate rise, spike, and fall of challenge across a sequence of levels that experienced designers use to keep a play session engaging. Evaluation frameworks used in the PCG community, such as expressive-range analysis and open benchmarking testbeds, have likewise concentrated on describing the diversity and quality of individual generated levels rather than on the shape of a difficulty trajectory across a run of levels.

This project addresses that gap by proposing a **Difficulty-Curve-Conditioned Procedural Level Generator**. Instead of generating levels independently from a single difficulty tag, the proposed system accepts a designer-authored difficulty curve – for example, “easy → spike → recovery → ramp” – as an explicit input, and generates a sequence of tile-based levels whose individual difficulties trace that curve point by point. The system is built around four cooperating components: a difficulty estimator that scores candidate levels using measurable proxies such as path length and enemy density; a trajectory-conditioned generative model that produces level content conditioned on a per-level target difficulty rather than a single global tag; an automated playability validator that guarantees every generated level is solvable before it is shown to a player; and a designer-facing curve editor that treats the difficulty trajectory itself as the primary creative object rather than as a by-product of content generation.

By moving difficulty conditioning from the level to the sequence, the proposed system operationalises pacing as a first-class, controllable property of a procedural generator. This is a deliberately narrower and more tractable target than open-ended, multi-modal PCG approaches that couple large language models, reinforcement learning, and symbolic constraint solvers,

while still addressing a concrete and recognised limitation of existing difficulty-aware generators: that they balance challenge within a single level far more often than they orchestrate a coherent trajectory across a run of levels.

The remainder of this report is organised as follows. Chapter 2 reviews the literature on dynamic difficulty adjustment, procedural level generation, level-segment assembly, and difficulty estimation that motivates and informs the design of the proposed system. Chapter 3 describes the environmental study, covering the hardware and software resources required to build and evaluate the system. Chapter 4 presents the system analysis, including the requirements, the shortcomings of existing systems, the proposed system with its advantages, and the functional, non-functional, and feasibility analysis of the project. Establishing this foundation is necessary before the review of related work that follows in the next chapter, which identifies the specific gap that this project fills.

CHAPTER 2

LITERATURE REVIEW

Procedural content generation (PCG) and experience-driven difficulty adaptation have seen substantial formalization within venues sponsored by the Institute of Electrical and Electronics Engineers (IEEE) and the Association for Computing Machinery (ACM). This chapter reviews a specialized corpus of peer-reviewed research spanning corpus-driven statistical sequencing, real-time player modeling, rapid Bayesian optimization, dungeon design, and systemic industry surveys. Together, these works establish the algorithmic foundation for modeling, generating, and validating adaptive game content.

2.1 Procedural Dungeon Generation for a 2D Top-Down Game

A foundational requirement for structural level generation is the ability to analyze and capture game space aesthetics, topology, and playability mechanics programmatically. Weeks and Davis explored this within top-down spatial structures, developing a procedural dungeon generator for 2D environments validated via search-based methods [1]. Their framework couples randomized structural layouts with a localized variant of the A^* search algorithm capable of navigating path obstructions, locks, and keys. By assessing specific topological features—such as optimal path lengths, dead ends, room density, and puzzle dependencies—the system extracts an unweighted complexity score to estimate static structural difficulty. This methodology demonstrates how automated agents can calculate structural difficulty metrics from level data, yet it treats generated dungeons as discrete, isolated instances rather than dynamic steps along an interconnected narrative pacing arc.

2.2 Experience-Driven PCG via Reinforcement Learning: A Super Mario Bros Study

When levels are generated sequentially rather than as isolated artifacts, optimizing for sustained player engagement requires moving beyond static topology metrics toward experience-driven reward structures. Shu et al. addressed this via the Experience-Driven Procedural Content Generation via Reinforcement Learning (EDRL) framework [2]. Framing the concatenation of level segments as a Markov Decision Process (MDP), they utilized Proximal Policy Optimization (PPO) paired with Generative Adversarial Networks (GANs) to dynamically assemble game sections. The agent’s reward landscape is driven by multi-objective experiential models designed to optimize simultaneously for moderated structural diversity, historical novelty, and absolute playability. While EDRL successfully steers macro-level sequences toward experiential targets, its generative policies are optimized against static generalized reward functions rather than being dynamically conditioned on a highly specific, designer-authored trajectory of target difficulty values.

2.3 Fast Game Content Adaptation Through Bayesian-Based Player Modelling

A core bottleneck in dynamic difficulty adaptation is the “cold-start” problem: human players possess highly subjective skill levels, yet traditional optimization methods require long, exhausting play sessions to accurately profile them. To achieve sample-efficient adaptation, González-Duque et al. introduced Fast Bayesian Content Adaptation (FBCA), which reframes player profiling as an optimization task solved via Gaussian Process Regression [3]. By shifting the objective from open-ended performance maximization to targeting a precise scalar value (e.g., a specific puzzle-completion time), their system converges on a personalized difficulty match within a minimal number of active interactions, vastly outperforming standard binary search baselines in Sudoku and roguelike environments.

2.4 Finding Game Levels with the Right Difficulty in a Few Trials Through Intelligent Trial-and-Error

Recognizing that generating content completely from scratch during live optimization is computationally risky, González-Duque et al. extended the boundaries of sample-efficient adaptation via an Intelligent Trial-and-Error (IT&E) architecture [4]. This paradigm bifurcates the adaptation process: an evolutionary algorithm (MAP-Elites) is first deployed offline to generate a diverse behavioral archive of level variants mapped against different simulated player profiles. Online, a Gaussian-Process-driven Bayesian optimizer samples from this pre-computed space, allowing the system to locate a structurally stable level matching a target win-rate threshold within fewer than five active player trials. Although IT&E provides a highly efficient mathematical framework for matching a player to a standalone scalar target difficulty, it operates on a trial-by-trial correction basis; it does not sequence a cohesive, continuous string of levels configured to execute a comprehensive, pre-determined macro difficulty curve over an extended playthrough.

2.5 Large Language Models and Games: A Survey and Roadmap

The macro-level trajectory of these generative technologies is mapped comprehensively in the survey by Gallotta et al., which establishes a structural roadmap for integrating generative models within interactive media [5]. Their analysis charts the evolution of content generation from deterministic, grammar-based systems to deep neural networks, highlighting a modern shift toward hybrid architectures. Specifically, the roadmap details the integration of large-scale statistical models with downstream constraint satisfaction networks and reinforcement learning loops to build multi-modal, co-creative design tools. Crucially, the authors note that the frontier of adaptive systems lies not in open-ended content generation, but in the development of rigorous, steerable constraint frameworks that allow human designers to specify exact experiential bounds—such as emotional trajectories, narrative tension arcs,

and explicit difficulty curves—which the algorithmic stack must reliably enforce.

2.6 Summary and Research Gap

Across the reviewed IEEE and ACM literature, three primary methodological limitations recur when mapped against the exact paradigms established by the core papers.

First, while algorithmic graph-based evaluations—such as those defined in *Procedural Dungeon Generation for a 2D Top-Down Game* [1]—are highly effective at executing localized A^* path checks and calculating localized layout metrics, they evaluate level configurations as discrete, isolated instances. They lack any mechanisms to coordinate these structures into a larger, continuous chronological string.

Second, online reinforcement learning models and data-driven optimization systems—such as the framework introduced in *Experience-Driven PCG via Reinforcement Learning: A Super Mario Bros Study* [2]—demonstrate that generation agents can be actively trained to concatenate sequential level pieces. However, these systems optimize content output against static, generalized reward targets (such as optimizing for broad structural fun or continuous historical novelty thresholds) rather than ingesting a highly detailed, multi-point target progression curve as an explicit conditioning vector.

Third, rapid personalization models—exemplified by *Fast Game Content Adaptation Through Bayesian-Based Player Modelling* [3] and *Finding Game Levels with the Right Difficulty in a Few Trials Through Intelligent Trial-and-Error* [4]—prove that combining offline evolutionary behavioral archives (MAP-Elites) with online Bayesian regression models allows a system to dynamically match a specific player to a bespoke challenge setting in very few physical updates. Yet, these architectures operate strictly on an isolated trial-by-trial correction basis; they are structurally built to seek a single static scalar target value (such as a target puzzle-completion time or a specific win-

rate percentage) rather than continuously generating an unfolding sequence of distinct levels designed to trace an entire pre-authored difficulty arc over an extended playthrough. Furthermore, as outlined in *Large Language Models and Games: A Survey and Roadmap* [5], contemporary procedural generation research lacks tightly integrated constraint satisfying structures capable of allowing creators to explicitly steer the structural composition of generated segments along exact, predetermined experiential and difficulty paths.

The proposed Difficulty-Curve-Conditioned Procedural Level Generator directly addresses the clear research gap identified across this core literature by introducing explicit, designer-authored, sequence-level difficulty-curve conditioning. By closing the functional distance between independent, per-level difficulty estimation and continuous, trajectory-based player-experience pacing, this project builds directly upon the established algorithmic capabilities of the field while addressing its primary architectural limitation.

Table 2.1: Comparative Summary of Core Literature

Study	Core Methodology	Key Contribution	Limitations
Weeks & Davis [1]	Search-based structural evaluation (A^*)	Procedurally generates 2D top-down dungeons; scores static structural complexity (path length, dead ends, room density)	Evaluates discrete, isolated dungeons; no pacing across levels
Shu et al. [2]	EDRL: PPO + GAN within an MDP formulation	Sequentially assembles Mario levels, optimizing a multi-objective reward for diversity, novelty, and playability	Static, generalized reward; no designer-specified difficulty trajectory
González-Duque et al. [3]	Fast Bayesian Content Adaptation (Gaussian Process Regression)	Rapidly profiles player skill by targeting a precise scalar value, minimizing required trials	Targets a single scalar value, not a difficulty curve
González-Duque et al. [4]	Intelligent Trial-and-Error: offline MAP-Elites + online Bayesian optimization	Matches a player to a stable level within fewer than five active trials	Trial-by-trial correction; no pre-authored macro arc
Gallotta et al. [5]	Systematic survey and roadmap	Charts the field's shift toward hybrid LLM/RL/constraint-satisfaction architectures	Identifies need for steerable control; no implementation

CHAPTER 3

ENVIRONMENTAL STUDY

This chapter discusses the environmental settings required to successfully develop, implement, and evaluate the Difficulty-Curve-Conditioned Procedural Level Generator. The environment of a project is important because it ensures that every component of the system – the difficulty estimator, the trajectory-conditioned generator, the playability validator, the designer trajectory editor, and the interactive demo interface – performs its function correctly and integrates smoothly with the others.

The environment entails both the hardware and software pre-requisites required for the system to operate successfully under realistic conditions. It covers the tools, frameworks, and platforms used throughout development, since the correct configuration of these elements directly affects the performance, training time, and reliability of the generative and validation components.

3.1 System Configuration

System configuration refers to the collection of hardware and software components necessary to develop and execute the system. The proposed system performs several computationally distinct operations – grid-based level representation, difficulty scoring, generative model training and inference, automated playability search, and interactive curve editing – so a properly configured environment is essential to keep processing time reasonable during both training and live demonstration.

3.1.1 *Hardware Requirements*

The hardware requirements outline the minimum resources necessary to train the difficulty estimator and the trajectory-conditioned generative model, run the playability validator, and serve the interactive demo without noticeable lag.

The key hardware requirements are detailed as follows:

- **Processor:** Intel® Core™ i5 Processor / AMD Ryzen™ 5 Processor or higher, to handle tile-based level generation, pathfinding, and validation tasks efficiently.
- **Memory:** Minimum 8 GB RAM (16 GB recommended), to support batch generation of level sequences and simultaneous execution of the playability validator during training and evaluation.
- **GPU (optional but recommended):** A CUDA-capable GPU with at least 4 GB of video memory, to accelerate training of the transformer- or GAN-based trajectory-conditioned generator. The system remains usable on CPU-only hardware for inference and for the smaller grid-based level format, at the cost of longer training time.
- **Storage:** 20–50 GB of free disk space, for storing the training corpus of levels, model checkpoints, generated level sequences, and evaluation logs.

3.1.2 *Software Requirements*

The software requirements specify the platforms, languages, and libraries used to implement level representation, difficulty estimation, generation, validation, and the designer-facing curve editor. Ensuring compatibility between these tools is important because the difficulty estimator, generator, and validator must exchange level data in a consistent format.

The key software requirements are detailed as follows:

- **Operating System:** Windows 10, Ubuntu 20.04 LTS, or macOS, to support cross-platform development and execution.
- **Programming Language:** Python 3.10 or higher, used for implementing the difficulty estimator, the generative model, the playability validator, and the supporting pipeline.
- **Version Control System:** Git, for tracking changes to the codebase, the generator's training configuration, and the human-rated calibration levels used by the difficulty estimator.
- **Required Python Libraries:**
 - PyTorch – a deep-learning framework used to implement and train the transformer- or GAN-based trajectory-conditioned level generator.
 - NumPy – used for efficient numerical manipulation of the grid-based tile representation of levels.
 - A* / pathfinding library (e.g. a lightweight custom or open-source A* implementation) – used by the playability validator to confirm that every generated level is solvable before it is surfaced to the designer or player.
 - Pygame – used to render generated levels for manual inspection and for the interactive demo, where a panel or tester can play through a generated sequence.
 - Flask (or an equivalent lightweight web framework) – used to serve the designer trajectory editor, through which a difficulty curve is drawn and submitted to the generation pipeline.
 - Matplotlib – used for expressive-range-style visualisation of generated level difficulty scores during evaluation.

3.1.3 *Software Requirements for Development*

Besides the execution specifications, certain tools are indispensable for development, testing, and maintenance of the system. These tools assist in writing and debugging the generator and validator code, managing the human-rated calibration dataset, and tracking experiments across model versions.

The key development software requirements are detailed as follows:

- **Operating System:** Windows 10, Ubuntu 20.04 LTS, or macOS, ensuring cross-platform compatibility.
- **Back End:** Python 3.10 or later, serving as the core implementation language for every module of the system.
- **IDE Used:** Visual Studio Code, used for development, debugging, and experiment management.
- **Version Control:** Git, for managing source code, model configurations, and the calibration-level dataset.
- **Development Tools and Frameworks:**
 - PyTorch – for training and iterating on the trajectory-conditioned generative model.
 - Pygame – for building and testing the interactive demo used to showcase generated difficulty curves.
 - Jupyter Notebook – for exploratory analysis of the difficulty estimator’s proxy metrics and for expressive-range-style evaluation of generated sequences.

3.2 *Software Specification*

This section describes the individual software components and technologies used to build the system in more detail. Each component performs a specific role within the pipeline – level representation, difficulty

estimation, generation, validation, or presentation – and the combination of these tools is intended to ensure a smooth, reproducible development process.

3.2.1 *Python 3.10*

Python was chosen as the primary programming language for this project due to its readability, its extensive ecosystem of machine-learning and game-development libraries, and the strong community support available for both deep learning and procedural content generation research. Level representation, difficulty scoring, model training, playability validation, and the web-based curve editor are all implemented in Python, allowing the different modules of the pipeline to interoperate without cross-language data conversion overhead.

3.2.2 *Visual Studio Code*

Visual Studio Code (VS Code) was selected as the primary Integrated Development Environment (IDE) used during development, testing, and maintenance of the system. It offers strong Python support, integrated debugging, Git integration, and extensions for Jupyter notebooks, which together streamline the iterative process of tuning the difficulty estimator and the generative model. Its cross-platform availability also makes it convenient for developing and testing the system across different operating systems.

3.2.3 *PyTorch*

PyTorch is used to implement the trajectory-conditioned generative model at the core of the system. Its dynamic computation graph and wide adoption in generative-modelling research make it well suited to experimenting with both transformer-based and GAN-based architectures for conditioning level generation on a per-level target difficulty rather than a single global tag. PyTorch is also used to implement or fine-tune the learned component of the difficulty estimator, where measurable proxies such as path length and

enemy density are calibrated against a small set of human-rated levels.

3.2.4 *Pygame*

Pygame is used to render the grid-based tile levels produced by the generator, both for internal debugging and for the interactive demo. Because the demo allows a panel or tester to play through a generated level sequence directly, rather than only viewing a dashboard of scores, Pygame's straightforward 2D rendering and input handling make it a practical choice for presenting the generated difficulty curve as a tangible, playable experience.

3.2.5 *A* Pathfinding / Playability Validator*

An A*-based solver is used as the automated playability validator described in the project's feature breakdown. Before any generated level is surfaced to a designer or player, the validator checks that a path exists from the level's start to its goal under the game's movement rules, filtering out broken or unsolvable generations. This mirrors the role played by A*-based completability checks in prior dungeon-generation and level-linking work, and is essential for guaranteeing that the difficulty scores produced by the estimator correspond to levels that can actually be played.

3.2.6 *Flask*

Flask is used to serve the designer trajectory editor, a lightweight web interface through which a designer draws a target difficulty curve across a sequence of N levels. Submitting the curve triggers the generation pipeline, which returns the matching sequence of levels for inspection or play. Flask's minimal footprint keeps this interface simple to develop and to demonstrate without introducing unnecessary infrastructure.

3.2.7 *Git*

Git is used as the version control system for the project's source code, model configuration files, and the small set of human-rated calibration levels used to tune the difficulty estimator. Version control is particularly important for this project because the calibration dataset, the generator's training configuration, and the difficulty-scoring weights are all likely to be revised iteratively as the system is evaluated, and Git allows these revisions to be tracked and, if necessary, reverted.

In this chapter, the hardware, software, and development environment required to build the Difficulty-Curve-Conditioned Procedural Level Generator have been discussed in detail. These configurations and tools are important in ensuring that level representation, difficulty estimation, trajectory-conditioned generation, and playability validation can all be carried out effectively and efficiently. With the execution environment established, the next chapter presents the system analysis, covering the requirements, limitations of existing systems, and the functional and non-functional specification of the proposed solution.

CHAPTER 4

SYSTEM ANALYSIS

System analysis is the process of structurally examining the proposed Difficulty-Curve-Conditioned Procedural Level Generator. The main purpose of this analysis is to understand the system’s functionality, processing flow, and the way its components interact to translate a designer-authored difficulty curve into a coherent, playable sequence of levels.

The analysis covers how a designer uses the system, starting from drawing a target difficulty curve across N levels, through generation of the corresponding tile-based level sequence, difficulty scoring, and playability validation, to the point where the sequence is presented for inspection or play. This analysis leads to a clear specification of the functional and non-functional requirements that the system must satisfy, and helps confirm that the proposed system directly addresses the pacing-related gap identified in the literature review, while remaining efficient, maintainable, and scalable to longer level sequences.

4.1 Requirements Analysis

The requirements analysis defines the broad functional and user requirements for a difficulty-curve-conditioned level-generation system. The system must be capable of accepting a designer-specified difficulty trajectory, generating a sequence of tile-based levels that follows that trajectory, scoring each generated level’s difficulty using measurable proxies, and guaranteeing that every level shown to the user is solvable. All requirements are specified so that they are clear, measurable, and attainable within the scope of a grid-based 2D level format such as a simple platformer or maze.

The system should also support a reasonable range of sequence lengths and difficulty-curve shapes (for example, easy, spike, recovery, and ramp segments) without requiring the designer to manually author individual levels. These requirements are necessary to design a reliable and intuitive system that generates pacing-aware level sequences without compromising accuracy, playability, or performance.

4.2 Existing System

Existing procedural level generators fall broadly into two categories. The first category consists of generators that condition each level on a single static difficulty label or on a random seed, using techniques ranging from n-gram statistical models trained on a level corpus to grammar-based and search-based generators evaluated through expressive-range analysis. These generators can be tuned to produce levels of a chosen style or difficulty, but each level is produced independently of the levels before and after it. The second category consists of dynamic difficulty adjustment systems, ranging from inventory-theoretic interventions and fuzzy-logic-guided agent behaviour to reinforcement-learning-based sequence generators such as Polymorph and the Experience-Driven PCG via Reinforcement Learning (EDRL) framework, and Bayesian approaches such as Fast Bayesian Content Adaptation and Intelligent Trial-and-Error. These systems adapt difficulty in response to observed player performance, either in real time or through iterative optimisation toward a single target value, but none of them accept an explicit, multi-point difficulty curve authored by a designer in advance as their primary creative input.

4.3 Limitations of Existing System

Although a substantial body of PCG and DDA technology already exists, the reviewed systems share the following drawbacks:

- Most level generators condition generation on a single global difficulty

tag, so producing a sequence of levels with a designed pacing requires generating individual levels separately and manually reordering or re-tuning them.

- Dynamic difficulty adjustment systems typically react to player performance level by level or encounter by encounter; they do not accept a designer-drawn difficulty curve spanning an entire run of levels as an explicit input before generation begins.
- Difficulty-aware generators such as Polymorph and EDRL optimise toward difficulty or experience proxies learned from play-trace data or reward functions, which makes it difficult for a designer to specify precisely where a difficulty spike or a recovery period should occur.
- Standard PCG evaluation frameworks, including expressive-range analysis and open benchmarking testbeds, are built around single-level quality, diversity, and controllability metrics, offering limited direct support for evaluating whether a generated sequence follows a designed pacing curve.
- Concatenating independently generated level segments into a longer sequence can break in-game structures or produce unsolvable levels unless a dedicated linking or repair mechanism is used, which many simple generators omit.

4.4 Proposed System

The proposed system is a Difficulty-Curve-Conditioned Procedural Level Generator that treats a designer-authored difficulty trajectory as the primary creative object, rather than as a by-product of independent level generation. The system accepts a difficulty curve drawn across N levels – for example, easy, spike, recovery, and ramp segments – and produces a sequence of tile-based levels whose individual difficulties trace that curve. It combines a difficulty estimator trained on measurable proxies such as path length and enemy

density, a generative model conditioned on a per-level target difficulty rather than a single global tag, and an automated playability validator that filters out unsolvable generations before they are surfaced. A designer trajectory editor and an interactive demo interface allow the curve to be drawn, the corresponding level sequence to be generated, and the resulting pacing to be experienced directly by playing through the generated levels.

4.5 Advantages of Proposed System

The proposed system offers the following advantages:

- Pacing becomes an explicit, controllable input rather than an emergent or unspecified property, allowing designers to author difficulty curves directly instead of manually adjusting individual level parameters.
- Difficulty estimation is grounded in measurable, interpretable proxies – such as path length, enemy density, required precision, and branching complexity – calibrated against a small set of human-rated levels, keeping the scoring process transparent and adjustable.
- Sequence-level conditioning closes the gap between per-level difficulty estimation, which is well studied in the literature, and designed player-experience pacing, which is comparatively under-served by existing evaluation frameworks and generators.
- Automated playability validation guarantees that every level in the generated sequence is solvable before it is shown, avoiding the broken or unsolvable output that can result from naïve concatenation of independently generated segments.
- The system reuses well-established, tractable PCG techniques – grid-based level representation and tile-based generative modelling – avoiding the added complexity of 3D engines or physics simulation while still producing a genuinely novel capability.

- The interactive demo allows a designer, tester, or evaluation panel to directly play through a generated sequence and feel the intended pacing, rather than only inspecting a dashboard of difficulty scores.

4.6 Functional Requirements

This section describes the core functionalities the system must provide to meet the requirements identified above. Functional requirements define the specific operations and behaviours of the system as it converts a designer-specified difficulty curve into a playable sequence of levels, and they form the basis for the system design presented in later stages of the project.

4.6.1 Level Representation Setup

The system must represent levels using a grid-based 2D tile format, such as a simple platformer or maze layout, so that generation remains tractable and each level can be validated for playability efficiently. This representation must be shared consistently across the difficulty estimator, the generative model, and the playability validator so that a generated level can be scored and checked without additional conversion steps.

4.6.2 Difficulty Estimator

The system must score a generated level's difficulty using measurable proxies, including path length, enemy density, required precision, and branching complexity. This scoring function may combine rule-based heuristics with a learned component, calibrated against a small set of human-rated levels, so that the resulting numeric difficulty score corresponds reasonably well to a human player's perception of challenge.

4.6.3 Trajectory-Conditioned Generator

The system must generate a sequence of levels using a generative model configured to ingest a continuous progression curve. To enforce macro-level pacing across the sequence rather than producing isolated level snapshots,

the architecture relies on an autoregressive transformer configuration or a GAN stack paired with a sequence-linking optimization loop. This generative pipeline is conditioned not on a single global difficulty value, but on an explicit target difficulty vector mapped across each progressive slot in the generation sequence. This ensures that the generated output forms a coherent structural progression that directly traces the designer's pre-authored curve, functionally bridging the gap identified in prior unconditioned sequence generation models.

4.6.4 Playability Validator

The system must run an automatic solver or pathfinder, such as an A*-based check, over every generated level before it is surfaced to the designer or player, confirming that the level is actually completable. Levels that fail this check must be filtered out or regenerated, guaranteeing that only solvable levels reach the output stage.

4.6.5 Designer Trajectory Editor

The system must provide a simple interface through which a designer draws a difficulty curve across N levels, in a manner similar to an emotion-arc editor but applied to game pacing. Submitting the drawn curve must trigger the generation pipeline and return the matching sequence of levels for review.

4.6.6 Interactive Demo Interface

The system must provide an interactive demonstration mode in which a drawn difficulty curve is converted into a generated run of levels that can either be auto-played by an AI agent or played directly by a tester or evaluation panel. This allows the intended pacing to be experienced first-hand rather than assessed only through numeric difficulty scores.

4.7 Non-Functional Requirements

The non-functional requirements specify the quality attributes the system must satisfy so that it remains effective, efficient, and usable while producing pacing-accurate output. These requirements complement the functional requirements above by constraining how well, rather than merely whether, the system performs its intended operations.

4.7.1 Scalability

The system must be able to generate difficulty-curve-conditioned sequences of varying length, from short demonstration runs to longer play sessions, without a disproportionate increase in generation time or a loss of consistency in how closely the generated sequence follows the target curve.

4.7.2 Usability

The designer trajectory editor must provide an intuitive, low-friction way to draw a difficulty curve and trigger generation, requiring no specialised knowledge of the underlying generative model or difficulty-scoring mechanism from the designer.

4.7.3 Reliability

The system must consistently produce levels that pass the playability validator and must handle malformed or extreme difficulty-curve inputs, such as very steep spikes, gracefully rather than failing outright, so that the generation pipeline remains stable during interactive demonstrations.

4.7.4 Maintainability

The system's architecture should keep the level representation, difficulty estimator, generative model, playability validator, and designer interface as loosely coupled modules, so that any one component – for example, swapping a transformer-based generator for a GAN-based one – can be modified or

replaced with minimal impact on the rest of the pipeline.

4.7.5 Performance

Level generation and playability validation should complete quickly enough to support the interactive demo, in which a designer draws a curve and expects to see or play the resulting level sequence within a short, demonstration-friendly timeframe.

4.8 Feasibility Study

This section discusses the feasibility of the proposed system from technical, operational, and economic perspectives. The system is considered viable because it builds on well-established, open-source PCG and machine-learning tools and deliberately targets a tractable grid-based level format rather than a full 3D game engine.

4.8.1 Technical Feasibility

This subsection evaluates the technical feasibility of implementing the proposed system. It relies on mature, widely used technologies, including Python, PyTorch for the generative model, A*-based pathfinding for playability validation, and Pygame or a lightweight web framework for the designer interface and demo, all of which are well documented and actively maintained. The chosen grid-based 2D level format keeps both generation and validation computationally tractable on standard development hardware.

4.8.2 Operational Feasibility

This subsection analyses the usability of the proposed system from the perspective of its intended users. Designers interact with the system primarily through the trajectory editor, drawing a curve and reviewing the generated sequence, which requires no programming or machine-learning

expertise. This keeps the day-to-day operation of the system accessible to game designers who are not necessarily technical specialists.

4.8.3 *Economic Feasibility*

This subsection analyses the cost-related aspects of the project. The system is built entirely from open-source libraries and frameworks, avoiding licensing costs, and its computational requirements are modest enough to run on standard consumer or academic hardware, with an optional GPU improving training time rather than being strictly required. This makes the system affordable and accessible for both individual developers and academic evaluation.

The analysis of the proposed Difficulty-Curve-Conditioned Procedural Level Generator was carried out by examining the system requirements, identifying the limitations of existing procedural level generators and dynamic difficulty adjustment systems, and evaluating the functional and non-functional properties of the proposed system. This analysis clarifies how the system is expected to behave as it converts a designer-authored difficulty curve into a validated, playable sequence of levels, and it establishes a clear foundation for the system design, implementation, and evaluation stages that follow.

CHAPTER 5

SYSTEM DESIGN

The design of the Difficulty-Curve-Conditioned Procedural Level Generator is elaborated on in this chapter, emphasizing the organization and structure of system components. The pipeline transitions single-level procedural generation into a holistic multi-level campaign engine capable of generating contiguous, playable levels tuned precisely to arbitrary designer pacing arcs. A modular design approach is used, orchestrating generative sequence models, A* pathfinding, and dynamic difficulty adjustment (DDA) algorithms to output a sequence of tile-based levels rather than treating each level as an independent artifact. This chapter also discusses how the modules interact with each other and how data flows from the designer, during trajectory setup, and from the player, during gameplay and telemetry collection.

5.1 *System Architecture*

As depicted in Figure 5.1, the architecture of the proposed system represents an automated pipeline for difficulty-trajectory-conditioned level generation and cumulative dynamic difficulty adjustment. The process begins when a designer authors a difficulty trajectory—such as a linear ramp, spike and recovery, or sawtooth wave—through a web-based curve editor spanning a sequence of levels.

This input is passed to the core generation pipeline, coordinated by a central pipeline module, which invokes the Trajectory-Conditioned Sequence Generator. Using a 2nd-Order Markov Chain trained on 31 authentic Super Mario Bros VGLC level maps, the generator produces candidate level grids

conditioned to match the designer's difficulty vectors. These candidate grids are subsequently routed to an automated playability validator utilizing a Robin Baumgarten A* pathfinding search agent. The validator guarantees that a path exists from the start to the finish within 200 seconds and enforces a difficulty variance constraint of less than or equal to 5%.

During active gameplay, the architecture integrates a Selective Cumulative Session DDA Engine. This engine evaluates player telemetry across all preceding levels played in the session using exponential decay weighting. The DDA adjustments are applied strictly to the next upcoming level, keeping micro-adjustments tightly bounded to within 10% of the designer's target curve. In parallel, a game session manager handles arcade states and life systems, while Java Swing-based interfaces execute interactive play and record full telemetry, including X and Y coordinate Death Zone data.

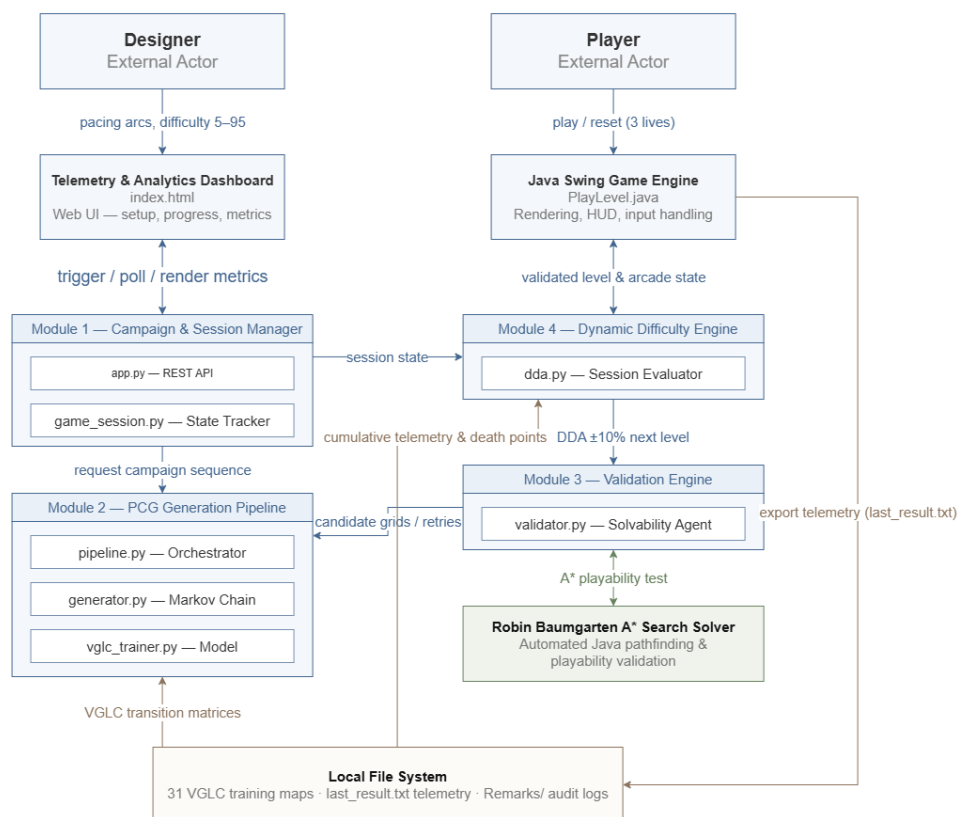


Fig. 5.1: Architecture Diagram of the PCG and DDA Framework.

5.2 Data Flow Diagram (DFD)

Figures 5.2 and 5.3 illustrate the data flow diagrams of the proposed system, showing the complete movement of data throughout the difficulty-curve setup, generation, validation, and real-time adjustment processes.

5.2.1 DFD Level 0

Figure 5.2 demonstrates the Level 0 Data Flow Diagram (DFD). A designer inputs a continuous difficulty trajectory via a web UI and receives a cohesive string of solvable game levels tracing that curve point-by-point. Players provide in-game input and telemetry data during play, and receive dynamically adjusted next-level challenges based on their cumulative historical performance.



Fig. 5.2: Data Flow Diagram Level 0.

5.2.2 DFD Level 1

Figure 5.3 presents the internal processing of the framework as a Level 1 DFD. The process begins when the web engine receives REST API calls containing the designer's requested difficulty curve. The pipeline script orchestrates generation by passing target difficulty vectors to the generator module, which samples 2nd-order column transition matrices generated from the training corpus. The candidate levels are passed to the validation module, which returns structural jump difficulty metrics and solvability booleans. If a level is unsolvable or deviates from the target difficulty by more than 5%, the generator automatically retries up to 50 times.

Once a sequence is validated, it is passed to the Java application for human or AI playback. The Java application writes telemetry to a result file, which is consumed by the DDA engine. The DDA calculates weighted cumulative

session performance indices and feeds micro-adjusted target metrics back to the pipeline to tune only the upcoming level. All telemetry, including localized death coordinates, is aggregated and returned to the web UI to be visualized on a unified 0-100 metric scale.

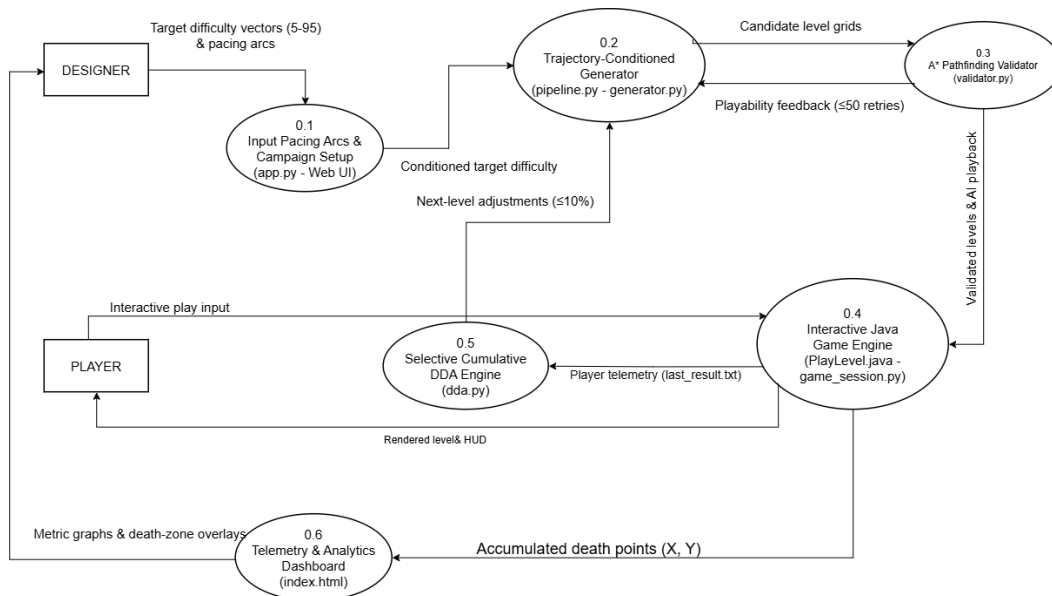


Fig. 5.3: Data Flow Diagram Level 1.

5.3 List of Modules

The PCG and DDA Framework is composed of the following core modules:

- Web Engine and User Interface
- Trajectory-Conditioned Sequence Generator
- A* Pathfinding Playability Validator
- Selective Cumulative Session DDA Engine
- Game Session and Arcade Manager
- Interactive Play and Telemetry Module

5.3.1 Web Engine and User Interface

This module features a Flask web engine providing REST API endpoints for real-time generation, Java play launching, and telemetry polling. The front-end consists of a modern glassmorphism web UI containing a designer difficulty curve inspector, a tile viewer, and Death Zone overlays. It utilizes a unified 0-100% metric scale to harmonize Chart.js wave-like graphs, target metrics, and designer curve input boxes. Additionally, a proportional loading progress modal displays step-by-step compilation progress from 0% to 100% in exact proportion to the levels successfully generated and validated.

5.3.2 Trajectory-Conditioned Sequence Generator

This module executes conditioned Markov sampling utilizing a 2nd-Order Markov Chain trained on 31 Super Mario Bros VGLC text levels. It generates contiguous column transitions to match predetermined designer difficulty vectors. The generator enforces authentic classic level structures by guaranteeing 8-step upward staircase intros, 8-step staircase endings with flagpole distance jumps, and automated castle walking sequences. Furthermore, it includes a stack block sanitizer to detect reward blocks stacked directly on top of solid obstacles, automatically shifting them up to jumping clearance heights or clearing them to ensure 100% hittable reward blocks.

5.3.3 A* Pathfinding Playability Validator

This module is responsible for testing solvability and measuring structural jump difficulty using an automated Robin Baumgarten A* pathfinding search agent. It tests whether Mario can navigate from the start of the level to the finish within 200 seconds, tracking jump heights, gap clearances, and hazard avoidance. If a candidate level is unplayable or violates the target difficulty variance by more than 5%, the module forces the generator to automatically retry the generation up to 50 times.

5.3.4 *Selective Cumulative Session DDA Engine*

The DDA module calculates weighted cumulative session performance indices to dynamically adjust level difficulty based on telemetry spanning all preceding levels played in the active session. It utilizes exponential decay weighting to combine overall session trends with the most recent player performance. Adjustments are applied strictly to the next upcoming level—leaving already played and unreached future levels untouched—and are tightly bounded to a maximum variance of 10% from the designer’s target curve. Generating an entirely new campaign sequence automatically resets this cumulative history and active DDA shifts back to 0.0%.

5.3.5 *Game Session and Arcade Manager*

Managed primarily by a game session Python script, this module manages the player arcade campaign state, life system, coin accumulation, and session progression tracking. It ensures player arcade level preservation, dictating that if a player loses 3 lives and resets the campaign, previously generated level layouts are preserved so players replay the exact same levels unlocked earlier.

5.3.6 *Interactive Play and Telemetry Module*

This module serves as the Java Swing entry point, executing human interactive play as well as live AI playback. It cleanly renders the remaining lives on the top-left in-game HUD by passing player lives and coins directly into the Java MarioWorld environment. The module captures complete telemetry outputs, prominently featuring cumulative Death Zone telemetry that records and renders all X and Y death points accumulated across multiple playthrough attempts per level map.

In this chapter, the overall system design of the framework is discussed, including the system architecture, the data flow diagrams, and the functionality of its various modules. With the system’s design and architecture established, the next chapter will cover the implementation

aspects, detailing the development of each module and how they integrate to achieve the intended functionality.

BIBLIOGRAPHY

- [1] M. Weeks and J. Davis, “Procedural Dungeon Generation for a 2D Top-down Game,” in *Proc. 2022 ACM Southeast Conf. (ACMSE)*, Virtual Event, USA, 2022, pp. 60–66.
- [2] T. Shu, J. Liu, and G. N. Yannakakis, “Experience-Driven PCG via Reinforcement Learning: A Super Mario Bros Study,” in *Proc. IEEE Conf. on Games (CoG)*, 2021.
- [3] M. González-Duque, R. B. Palm, and S. Risi, “Fast Game Content Adaptation Through Bayesian-based Player Modelling,” in *Proc. IEEE Conf. on Games (CoG)*, 2021.
- [4] M. González-Duque, R. B. Palm, D. Ha, and S. Risi, “Finding Game Levels with the Right Difficulty in a Few Trials through Intelligent Trial-and-Error,” in *Proc. IEEE Conf. on Games (CoG)*, 2020, pp. 503–510.
- [5] R. Gallotta, G. Todd, M. Zammit, S. Earle, A. Liapis, J. Togelius, and G. N. Yannakakis, “Large Language Models and Games: A Survey and Roadmap,” *IEEE Transactions on Games*, 2024.